DINQ

DINQ Technical Report • dinq.me

An Interactive Agent for Requirement-Driven Candidate Sourcing

Yuanpeng He[1], Fangjing Li[2], Xiangyu Ru[3], Kexin Sun[4], Kun Yang[3], Lijian Li[5], Chi-Man Pun[5], Qingsong Wen[6], Wenpin Jiao[1], Mingkai Guo[7], Yirong Feng[7], Daiheng Gao[7,8], Zhi Jin[9]

[1]Peking University [2]Beijing Jiaotong University [3]Beihang University [4]Beijing University of Technology

[5]University of Macau [6]Squirrel Ai Learning [7]DINQ.inc [8]University of Science and Technology of China [9]Peking University / Wuhan University

heyuanpeng@stu.pku.edu.cn, lifangjing@bjtu.edu.cn, zy2514120@buaa.edu.cn, Skx7087@emails.bjut.edu.cn, 22377023@buaa.edu.cn, yc57484@umac.mo, cmpun@umac.mo, qingsongedu@gmail.com, jwp@pku.edu.cn, keith@dinqlabs.com, elonf@dinqlabs.com, sam@dinqlabs.com, zhijin@pku.edu.cn

***Abstract*—Finding people from a natural-language description ("ML engineers transitioning to research roles in biotech") is increasingly delegated to LLM agents and framed as information retrieval. We argue that it is fundamentally a requirements engineering task: such a request is an under-determined requirement with implicit constraints, many valid answers, and no acceptance criterion, so useful answers require eliciting, validating, and verifying the requirement before search can matter. We present ⌑ DINQ TalentScout, to our knowledge the first interactive, requirements-driven candidate-sourcing agent (it elicits, validates, retrieves, and verifies a vague people-request into a justified slate through bounded elicitation, workflow templates, a two-stage commit protocol, and bidirectional termination guards) and TalentTrace, a benchmark that runs the requirements lifecycle (criteria-anchored validation, multi-model evidence-grounded oracle construction, and cost-aware verification). Across 21 systems and all 691 requirements, ⌑ DINQ TalentScout dominates breadth (100% coverage at 2.5× the yield) and is *near-orthogonal* to the field, with 90% of the people it returns are surfaced by *none* of 20 strong LLM-plus-web baselines combined. Beyond breadth, an evidence-grounded judging of every system shows ⌑ DINQ TalentScout *recalls* the most relevant real people: 0.241 of the union pool, 1.9× the next system, with a bootstrap 95% interval disjoint from every baseline. ⌑ DINQ TalentScout is thus the strongest *sourcing* engine (the deepest real, reachable candidate pool), while precision-ranking LLMs serve as complementary verifiers.**

***Index Terms*—requirements elicitation, requirements validation, LLM agents, people search, expert finding, benchmark, test oracle**

I. Introduction

Finding the right *people* from a vague description is routine and high-stakes in hiring, staffing, expert finding, and team formation. Consider the request: *"find ML engineers transitioning to research roles in biotech."* Today an expert answers it by hand, translating an under-specified wish into Boolean queries and skimming hundreds of profiles, slowly and irreproducibly. It is tempting to hand the request to an LLM web agent, yet for this request a strong GPT-class web agent returns only *five* people, none of whom overlap the 46 a requirements-driven agent finds. The web is not missing these people; the system is asking the wrong question.

The difficulty is not search. Classical expertise retrieval and recent LLM recruiting tools assume the request is *already a well-formed query* and optimize ranking for it [1]–[3]; they screen or rank candidates against a *finished* specification rather than help *author* one. A vague people-request is not a query: it is an *under-determined requirement*: it leaves constraints implicit (how senior? how many? what counts as "transitioning"?), admits many valid answers rather than one, and supplies no acceptance criterion against which a returned person can be checked, and *no prior system authors that requirement in the first place.*

Our key insight is that candidate sourcing is, at its core, a *requirements engineering* (RE) problem: the bottleneck is eliciting, validating, and verifying the requirement, not retrieving against it [4]–[6]. We present ⌑ DINQ TalentScout, to our knowledge the **first interactive, requirements-driven candidate-sourcing agent**: it *elicits* latent constraints through a bounded clarification interview, *validates* the requirement's well-formedness and kind, *retrieves* over a hybrid people-universe, and *verifies* each delivered person against the requirement with grounded evidence, through one principle applied to four sources of agent divergence (§IV). To the best of our knowledge, no published system casts people-search as RE or authors a people-requirement interactively from a vague request.

To evaluate this setting, where no benchmark for the task exists, we also build TalentTrace, which itself performs the RE lifecycle (generating and validating requirements, constructing

evidence-backed acceptance oracles, and verifying satisfaction) to evaluate sourcing agents at scale. On its 691 requirements and 21 systems, ⌑ DINQ TalentScout is not merely the highest-coverage and highest-yield agent but *near-orthogonal to the entire field*: 90% of the people it returns are surfaced by *none* of 20 strong LLM + web baselines combined. Grading every delivered person against evidence then settles the harder question: are these the *right* people? ⌑ DINQ TalentScout *recalls* the most relevant real people of any system (0.241 of the union pool, 1.9× the next, bootstrap-significant): it is the deepest *sourcing* engine, while precision-ranking LLMs are complementary *verifiers*.

In summary, this paper makes four contributions:

- **⌑ DINQ TalentScout, the first interactive, requirements-driven candidate-sourcing agent** (to our knowledge): it elicits, validates, retrieves, and verifies an under-determined people-request into a justified slate, organized by a single design principle that converges an LLM agent's divergence on four axes (§IV).
- A **reframing** of candidate sourcing as requirements engineering (the conceptual lens that makes the agent principled and brings the test-oracle problem to people-finding), which is itself a novel problem formulation (§III).
- **TalentTrace**, a benchmark that performs the RE lifecycle (validation, evidence-grounded oracle construction, cost-aware verification) to evaluate sourcing agents at scale (§V).
- An **empirical study** over 21 systems and all 691 requirements (§VII): ⌑ DINQ TalentScout is *near-orthogonal* to the field (90% of the people it returns are surfaced by none of 20 baselines), robust across difficulty, type, and language, and, under evidence-grounded judging, *recalls the most relevant real people of any system* (1.9× the next, bootstrap-significant), framing ⌑ DINQ TalentScout as a deep *sourcing* engine and precision-ranking LLMs as complementary *verifiers*.

II. Background and a Motivating Example

A sourcing request is *closed* when its constraints define a determinate, enumerable answer set ("the program co-chairs of ICSE 2025") and *open* when they allow many equally valid answers with no practical enumeration ("ML engineers transitioning to research roles in biotech"). This distinction changes both what a *correct* answer is (a set vs. a ranked pool), and how the requirement must be elicited and validated. Open requirements dominate recruiting, expert finding, and due diligence, yet lack a single "gold answer," placing them outside standard retrieval benchmarks [1].

Consider one real benchmark requirement, *"Find ML engineers transitioning to research roles in biotech in the Silicon Valley or Boston area, active in 2024–2026"*, with at least six constraints (role, a *transition* predicate, industry, two regions, a recency window) plus implicit choices (how senior? how many?). For exactly this requirement, ⌑ DINQ TalentScout returns 46 named, evidence-backed candidates (e.g., R. Nishihara, E. Bernard, C. Wong-Fannjiang, all genuine ML-to-biotech-research movers), whereas a strong GPT-class web agent returns only 5 (A. Lu, F. Rubbo, ...), with *zero* overlap. The gap is not search coverage but two *requirements* failures of the baselines: *(i) ungrounded candidates* whose cited evidence, when fetched, does not support the claimed role (21–27% are outright unsupported, §VII), and *(ii) constraint-violating matches* that are lexically on-topic (senior ML researchers in biotech) yet break a binding constraint (always researchers; wrong region). The first misses *verification*, the second *elicitation*, so the binding difficulty is requirements-level, not query-string-level, motivating the co-design of §IV and §V.

III. Candidate Sourcing as Requirements Engineering

Treating sourcing as retrieval (encoding the request as a query and ranking profiles by similarity) confuses a symptom with the problem. Following Zave and Jackson [4], a requirement is an *optative* predicate over phenomena *in the world*, not in the machine. A sourcing request ("a senior NLP researcher who has shipped production systems") is such a predicate R over persons, and its deliverable is not a single artifact but the *extension* $A(R) = \{p : R(p)\}$, the ranked set of everyone who satisfies it. Satisfaction therefore conjoins two relations: *set membership* (do delivered people belong to the intended extension?) and *acceptance grounding* (is each membership decision entailed by evidence?). The lifecycle is elicit/validate → operationalize (S, W) → verify (oracle O, with adequacy $W \wedge S \models R$); four consequences, each a classical RE concern, follow.

(i) Elicitation. R is under-specified (with tacit constraints, unstated trade-offs, and unstated open/closed scope), so it must be elicited; in information-seeking terms it is an *anomalous state of knowledge* [7], the formal basis for interaction. **(ii) An oracle.** Deciding $p \in A(R)$ from evidence requires an *acceptance oracle* [8]; for *open* requirements no answer key exists (they are *non-testable* [9]), so the oracle is *derived, partial, probabilistic*, and *abstention* on under-evidenced candidates is correct oracle behaviour, not a coverage gap. **(iii) Verifiability binds.** By the standards' own criterion [10], [11], a request that cannot be operationalized as an evidence-checkable predicate is not yet a well-formed requirement; elicitation *manufactures* testability. **(iv) Criteria are requirements.** Each rule inferring a property from evidence ("first-author ACL papers ⇒ NLP expertise") is a domain assertion that "should be validated before it is used" [4]; in the WRSPM model [12], with W validated domain knowledge and S the acceptance spec, the *adequacy* obligation $W \wedge S \models R$ gives the oracle a requirement's full quality burden: constructing it recapitulates the RE lifecycle one level up.

These obligations partition the architecture along Boehm's validation/verification line [13]. *Validation* ("did we elicit and operationalize the *right* requirement?") is served by bounded elicitation (§IV-C) and criteria-anchored debate (§V-B). *Verification* ("are delivered people grounded *against* the spec?") is served by the evidence-grounded oracle and commit–defer scoring (§V-D). Open vs. closed is itself a validation property: a

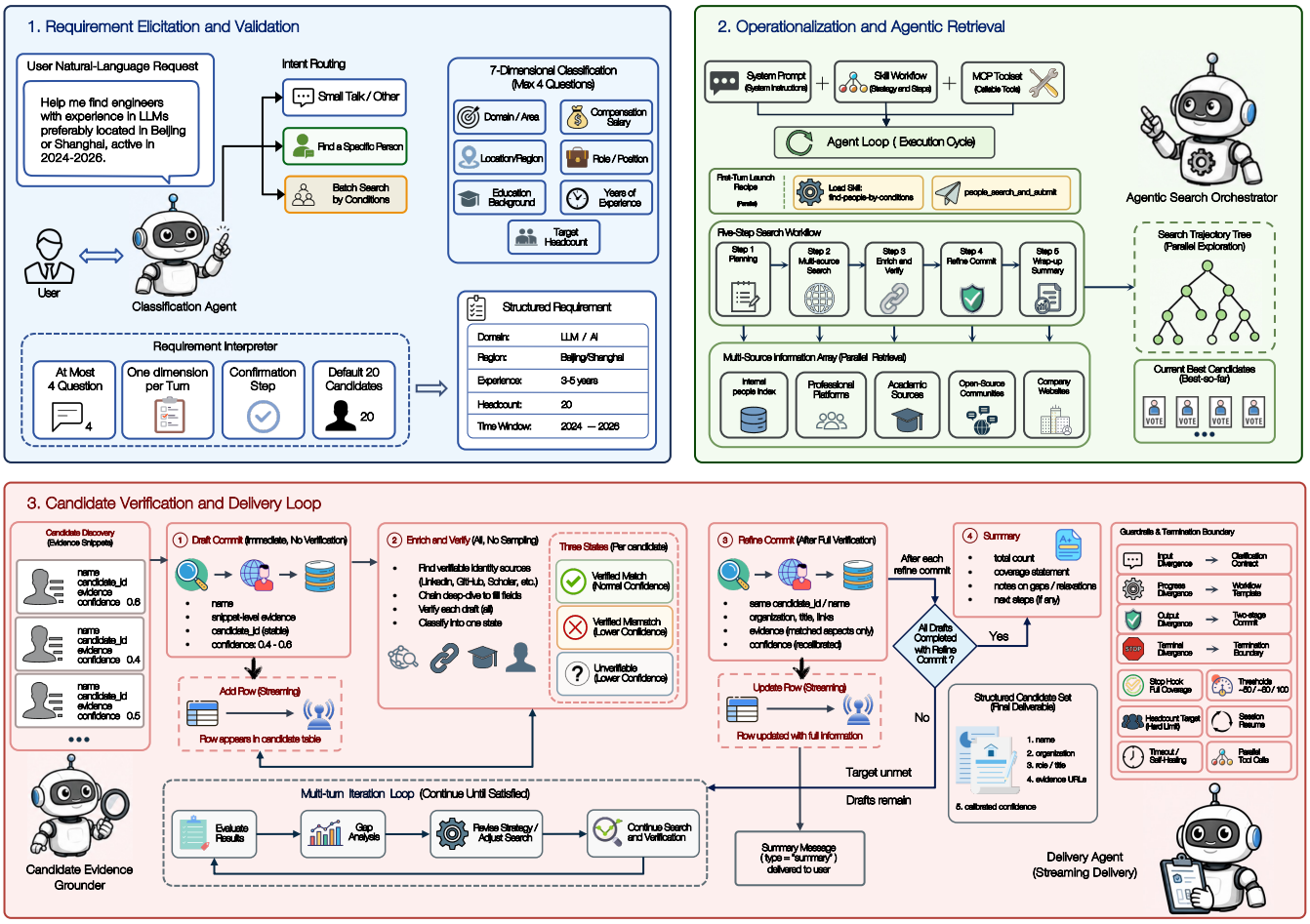

Fig. 1. **Architecture of ⌑ DINQ TalentScout, the requirements-driven sourcing agent.** One requirements constraint tames each axis on which a tool-using LLM agent diverges. *(1) Requirement elicitation and validation:* a three-way intent router and a bounded seven-dimension clarification interview ($\leq$4 questions, then a confirmation turn) convert a vague request into a structured, confirmed requirement. *(2) Operationalization and agentic retrieval:* a fixed workflow (a retrieval kickoff recipe and a five-step skill) searches a multi-source people index (internal index, professional, academic, open-source, and company sources) in parallel. *(3) Candidate verification and delivery:* a two-stage commit grounds each delivered candidate in evidence and resolves it into one of three states, streamed to the user under explicit guardrails and a bidirectional termination boundary.

closed requirement has a finite, enumerable acceptable set with a total, decidable membership test; an open one ranges over an unbounded population, so its oracle is necessarily partial and graded: an *oracle, not an answer key*. The model also names the two failure modes our study separates: a *validation failure* returns the extension of the wrong R (recall-oriented, "missed the right people"), and a *verification failure* admits an ungrounded p (precision-oriented, "cannot justify a delivered person"). Their error modes are orthogonal, which is why naive query-string people search simultaneously *misses and hallucinates*, and why the fix belongs at the *requirements* level, not the query string.

IV. ⌑ DINQ TalentScout: An Interactive Requirement-Driven Sourcing Agent

A. Problem Formulation

We study *open-ended candidate sourcing*: given a natural-language request (e.g., the example of §II), the system must return a *bounded, ranked, evidence-backed list of real people* who satisfy it. As argued in §I, the request is an *under-determined requirement*; a usable answer therefore requires the classic RE activities (*eliciting* implicit constraints, *validating* that the requirement is well-formed and answerable, and *verifying* each delivered person against it) before retrieval is useful.

Formally, let r be the user's initial utterance. ⌑ DINQ TalentScout first elicits a *structured requirement* $R = (D, K, n)$, where D is a set of constraint dimensions with values (domain, location, seniority, ...), K is the *kind* of the requirement (open-ended vs. closed), and n is a target cardinality. It then produces a candidate set $C = \{c_1, \ldots, c_m\}$, $m \leq n$, where each c_i includes a name, an organization, a role, supporting evidence URLs, and a calibrated confidence. The engineering objective is to steer an autonomous large-language-model (LLM) agent from the under-determined r to a C that is *bounded* (m is controlled, the loop terminates), *verifiable* (every c_i is backed by checkable evidence rather than a hallucinated assertion), and *structured* (machine-mergeable rows, not prose).

B. Design Principle: Four Divergences, Four Constraints

Figure 1 sketches the full loop. Left unconstrained, an LLM agent that can call search and scraping tools over many turns is *divergent* along four orthogonal axes, each matching a repeated failure mode: (i) at the **input**, the requirement is under-determined, so the agent guesses missing constraints; (ii) during the **process**, search is exploratory and irreproducible, so runs wander; (iii) at the **output**, fast results conflict with complete, verified fields; and (iv) at **termination**, the agent either stops too early (before verifying its own candidates) or never stops (spinning on a query with no hits). ⌑ DINQ TalentScout applies one principle four times: *converge each divergence with an explicit engineering constraint*, realized by the four-stage loop of Fig. 1 (elicit, validate, retrieve, verify). The four constraints are layered and orthogonal: a decision layer (system prompt + skills), a capability layer (tools), an execution layer (the agentic loop and runtime guards), and a delivery layer (incremental streaming), so each can change without disturbing the others; the rest of this section details each.

C. Eliciting the Requirement (Clarification Contract)

The first constraint treats the under-determined input as a *bounded elicitation interview*. A system prompt fixes an *output contract*: every user-facing turn is a single JSON envelope {`type`, `content`, `option`}, language-locked, with internal tool names masked, and a three-way *intent router* (specific-person / by-conditions / other). For the open-ended branch, ⌑ DINQ TalentScout runs a *progressive, one-dimension-per-turn* elicitation over seven requirement dimensions (domain, compensation, location, role, education, experience, and head-count), each surfaced as a question with quick-reply options. Two hard limits keep it terminating and non-fatiguing: *at most four* questions (then a confirmation turn, even if dimensions remain unknown) and one dimension per turn, with a missing head-count defaulting to twenty. This casts elicitation as *information collection under a hard budget* (the trade-off human analysts make in time-boxed interviews [14], [15]) and the confirmation turn is the user's last chance to correct the requirement before any retrieval begins.

D. Operationalizing the Requirement (Workflow Template)

The second constraint replaces free exploration with a *fixed workflow*. A *retrieval-kickoff recipe* runs the first post-confirmation turn: in one turn the agent loads the skill *and* issues a combined "search-and-submit" call against an internal index (limit = head-count), so the first candidates appear within seconds, eliminating a "search→inspect→submit" round trip. Subsequent turns follow a five-step skill: (1) *plan* sources; (2) *search* broadly in parallel, drafting each surfaced person immediately; (3) *enrich and verify every* draft (full coverage, no sampling); (4) *commit* (§IV-E); (5) emit `summary` only after the target is met and every draft refined. Evidence is gathered by *chaining*, where one tool's name/URL feeds the next (LinkedIn via a dedicated channel, other URLs via a reader), turning a thin snippet into checkable evidence.

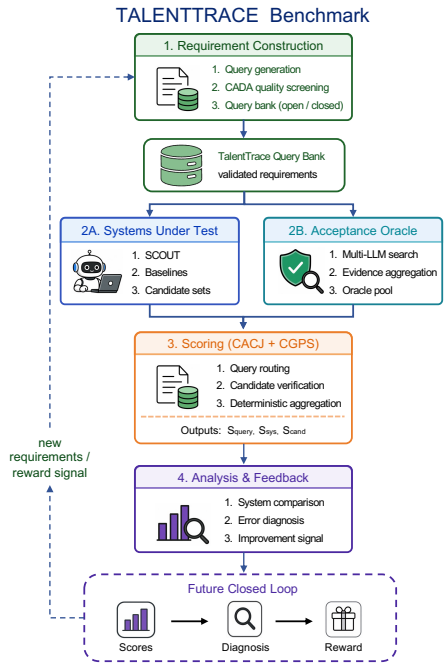

Fig. 2. **The TalentTrace evaluation pipeline.** Requirements are generated and quality-screened by the CADA debate jury into an open/closed query bank of validated requirements. Systems under test (⌑ DINQ TalentScout and baselines) return candidate sets while a multi-LLM, evidence-aggregating oracle builds acceptance pools; CACJ routes each requirement and CGPS verifies and deterministically aggregates candidates into per-system scores. Analysis diagnoses failures and, as future work, closes the loop as a reward signal (§VIII).

E. Decoupling Speed and Quality (Two-Stage Commit)

The sole output channel is a `submit_candidates` tool, and every candidate is submitted *twice*. The *draft* fires the instant a person appears (name, snippet evidence, a stable `candidate_id`, confidence in $[0.4, 0.6]$) and the front end appends a row immediately. The *refine* repeats the same `candidate_id`/name, fills the now-mandatory organization and role, attaches links, and re-calibrates confidence; the front end merges on `candidate_id`. This *decouples* the conflicting output goals (drafts buy sub-second responsiveness, refines buy field-complete grounded quality) without double counting. Verification resolves each candidate as verified-match, verified-mismatch, or unverifiable; crucially, evidence text records *only matched aspects*, doubt being expressed solely through lower confidence.

F. Bounding Termination (Termination Boundary)

The final constraint makes *stopping* a bidirectional concern. Against stopping *too early*, a *full-coverage guard* keeps the agent working while any candidate is draft-only: a stop-hook is designed to force another round (up to three), and in the shipped runtime this is enforced by the prompt and skill plus a fallback retry that re-submits searched-but-unsubmitted candidates (the hook intercept is currently inactive, §IX). Against *never stopping*, the agent self-evaluates against cumulative tool-call thresholds: at ∼50 calls with no hits it must change strategy,

Algorithm 1 CADA: Criteria-Anchored Debate Adjudication

Require: item x; criteria K; debaters D; arbitrator A; threshold λ
Ensure: verdict v and judgment path
for each debater $d \in D$ **do** ▷ independent, parallel
  $s_d \leftarrow$ binary scores of K on x; $(\text{stance}_d, \text{conf}_d) \leftarrow d(x)$
end for
$w \leftarrow$ confidence-weighted agreement of $\{\text{stance}_d\}$ ▷ moderator
if stances unanimous **and** $w \geq \lambda$ **then**
  return (CONSENSUS($\{s_d\}$), `early_exit`) ▷ skip A
end if
$C \leftarrow$ criteria on which $\{s_d\}$ disagree ▷ contested
return $(A(x, \{s_d\}, C)$, `meta_judge`$)$ ▷ rule on C

at 80 narrow, at 100 halt and report honestly. Together these bound termination in an interval that is neither premature nor unbounded, which we exploit when diagnosing failures (§VII).

G. Runtime

⌑ DINQ TalentScout drives the Claude Agent SDK [16] as a subprocess and streams events over Server-Sent Events so candidates appear and update live; the runtime adds session resumption, stuck-detection with reconnect-and-resume, per-session credential isolation, and bounded tool concurrency. This scaffolding supports stable execution; the section's contribution is the four-constraint principle, of which the full-coverage guard (a runtime mechanism enforcing a process property) is the clearest cross-layer example.

V. TalentTrace: Constructing and Evaluating Sourcing Requirements

Evaluating an agent like ⌑ DINQ TalentScout requires more than queries and answer keys. Because each request is an under-determined requirement (§IV-A), the evaluation must itself do requirements engineering: *generate* diverse candidate requirements; *validate* which are well formed and determine their *kind* (open vs. closed); build an *acceptance oracle* for each, where open requirements require graded pools rather than single answers; and *verify* delivered candidates against that oracle while controlling fabricated evidence. TalentTrace implements these four activities (Fig. 2) with query generation, a criteria-anchored debate procedure CADA used twice (first to *admit* requirements by quality and then, as CACJ, to *classify* their kind), a ground-truth (GT) pipeline, and the CGPS (Commit-Gated People Scoring) scorer; we detail each below.

A. Generating a Population of Requirements

A useful benchmark needs requirements that are broad, reproducible, and not merely hand-curated. TalentTrace therefore *generates* candidates in two families: *open* requirements, which admit many valid people, and *closed* requirements, whose answers form a finite enumerable set. Diversity is controlled by rotation over four domain axes, temperature jitter (open 0.72, broad-open 0.78, closed 0.55), and injection of existing queries as negative examples to suppress paraphrase duplication. Closed requirements are further balanced across six task types (verification, cross-verification, ranking, deduplication, recency, and niche-stability) and three difficulty levels by a deterministic cycle; after generation, the task-type field is force-overwritten to guarantee the intended distribution. From 1,500 generated requirements (840 open, 660 closed), validation (§V-B) admits 1,007 (691 open, 316 closed).

B. Validating Requirements with Criteria-Anchored Debate

The central methodological problem is deciding which generated requirements are good enough to evaluate. A single LLM judge is too unstable for this gate: in our data, one GPT-class judge rates 86.8% of closed requirements as perfect (mean $4.54/5$), while a Claude-class judge averages 3.62, so model-specific optimism would silently set the benchmark. We instead use CADA (*Criteria-Anchored Debate Adjudication*, Alg. 1). Three *heterogeneous* LLMs (Claude-, Gemini-, and GPT-class) first score a checklist of *binary criteria*, then take a stance. A deterministic moderator checks consensus; only disagreement triggers a fourth arbitrator, which rules on contested criteria (scores read before prose, clamped to $[\min, \max]$, conservative default). Scoring criteria *before* verdicts makes each decision auditable and localizes disagreement.

CADA answers two validation questions with the same machinery.

(1) Quality admission. The first question is whether a requirement is well formed enough to enter the benchmark. Seven binary criteria map almost one-to-one to the classical requirements-quality attributes of IEEE 830 and Wiegers [10], [17] (Table I): a requirement is admitted only if it is unambiguous, feasible, internally consistent, *verifiable*, appropriately scoped, temporally valid, and free of fabricated premises. Five graded dimensions (overall value, clarity, benchmark-fit, retrievability, specificity) yield a 1–5 quality score; requirements scoring ≥ 3.5 are admitted. *Specificity* is lowest for both families but for *opposite* reasons: open requirements risk being too narrow, closed ones too broad, so the rubrics penalize opposite failures.

(2) Kind classification (CACJ). The second question is how the admitted requirement should be scored. CACJ (*Criteria-Anchored Cascade Judgment*) decides whether it is OPEN or CLOSED: whether its constraints determine an enumerable answer set or only an open pool. This is a requirements-*boundedness* judgment. Six binary criteria (`entity_constrained`, `temporal_bounded`, `geographic_or_org_bounded`, `quantity_constrained`, `population_finite`, `count_below_threshold`) feed the decision, with one operational cutoff: if more than MAXN $= 20$ valid results are expected, the requirement is OPEN regardless of other signals. Because the three voters usually agree, a confidence-weighted moderator *early-exits* without arbitrator on 95.3% of closed and 97.5% of open requirements, reserving expensive adjudication for the contested $\sim$ 3–5%. This commit–defer design (cheap when easy, expensive only when ambiguous) recurs in scoring (§V-D).

C. Constructing the Acceptance Oracle (Ground Truth)

Closed requirements can use small enumerable answer sets; open requirements, which we evaluate, need an oracle that

TABLE I
Admission criteria in CADA. The seven binary tests are exactly the classical requirements-quality attributes, applied to a people-search requirement.

CADA criterion	RE attribute	Violated when…
`intent_clear`	Unambiguous	intent is vague
`search_exec`	Feasible	no realizable search path
`type_consistent`	Consistent	stated type vs. intent clash
`constraint_verif`	Verifiable	a constraint is uncheckable
`scope_bounded`	Appropriate	too broad or too narrow
`temporal_valid`	Valid/current	time window ill-posed
`no_halluc_entity`	Correct	names a non-existent entity

captures many acceptable answers. TalentTrace therefore builds, for each open requirement, a *graded, evidence-backed pool of real people*. Three heterogeneous search models (Gemini-, Claude-, and Grok-class) independently answer the requirement, each with a live web-retrieval tool loop (up to ten rounds of neural search and page-fetching), so every proposed person is tied to source URLs. A *cross-family* GPT-class judge scores each candidate on source reasonableness, evidence depth, and overall usefulness and, crucially, *fact-checks* it by fetching the cited page and emitting a `fact_supported` value (1 if the page corroborates name, organization, and role; partial credit down to 0 for fetch failures or contradictions). Finally, a synthesis model deliberately *not* from the generator families merges the three candidate sets into a consensus pool ranked by cross-model support, evidence quality, and host diversity. Consensus is over *evidence*, not surface strings: raw three-way name overlap is only ≈ 0.02 Jaccard, so candidates are promoted when multiple models *independently corroborate the same person with checkable sources*, not when they emit identical text. If models neither agree nor ground the candidate, the oracle remains `null`. Over the evaluation set this yields 203 requirements with a mean pool of 44.6 people (range 6–67), mean synthesis reliability 0.60, full three-generator coverage, and total construction cost $347.77.

D. Verifying Satisfaction (CGPS)

Given a system's delivered candidates and the oracle, CGPS (Alg. 2) verifies satisfaction with the same commit–defer discipline. CADA (re-used) fixes the requirement kind and required constraints at the *query* level; at the *candidate* level a cheap relevance gate *commits* a verdict from each delivered person's structured fields and *defers* the expensive evidence fact-check to a calibration subset, the cost-aware analogue of the agent's own commit–defer, since fact-support is noisy for concise profiles (§IX). Crucially, CGPS scores *recall over a method-neutral pool* rather than a per-candidate composite: an earlier per-candidate usefulness score rewarded verbose prose and a good-rate penalized depth (§VII-D), so we instead measure how many relevant real people a system recalls from the union of *all* systems' returns: a deterministic, label-only metric that cannot drift with judge verbosity.

For each candidate, the CGPS judge emits a structured verdict: a primary issue bucket (OFF-TOPIC, THIN-SOURCES, BROKEN-URL, LIKELY-HALLUCINATION, or NONE) plus

Algorithm 2 CGPS: Relevance-Grounded Recall over a Neutral Pool

Require: systems $\mathcal{S}$; delivered candidates $C_{s,q}$ for system s, query q; judge J
Ensure: per-system recall, reachable yield, yield/req., bootstrap 95% CIs
for $s \in \mathcal{S}$, query q, candidate $c \in C_{s,q}$ **do**
  $b \leftarrow J(c)$ ▷ cheap relevance gate; fact-check deferred
  $\mathrm{rel}(c) \leftarrow [\, b \neq \text{OFF-TOPIC} \,]$; $\mathrm{reach}(c) \leftarrow \textsc{HasUrl}(c)$
end for
$R_{s,q} \leftarrow \{\, c \in C_{s,q} : \mathrm{rel}(c) \,\}$ ▷ relevant real people
$U_q \leftarrow \bigcup_{s \in \mathcal{S}} R_{s,q}$ ▷ method-neutral union pool
for each system s **do**
  $\mathrm{recall}_s \leftarrow \sum_q |R_{s,q} \cap U_q| \,/\, \sum_q |U_q|$
  $\mathrm{reach}_s \leftarrow \sum_q |\{c \in R_{s,q} : \mathrm{reach}(c)\}|$; $\mathrm{yield}_s \leftarrow \frac{1}{|Q|} \sum_q |R_{s,q}|$
  $\mathrm{CI}_s \leftarrow \textsc{Bootstrap}_{500}(\mathrm{recall}_s)$ ▷ resample queries
end for
return systems ranked by recall_s with 95% CIs

usefulness, evidence-depth, and source-reasonableness ratings, all reduced *deterministically* to scores (the LLM emits only labels). A candidate counts as a *relevant real person* when it is not OFF-TOPIC: relevance depends on name, organization, and title, not evidence thickness, so concise structured rows from a deep sourcing engine are not penalized. Per system, CGPS then reports, over the union pool of relevant real people, *recall* and *reachable yield* (relevant candidates with a fetchable URL) for sourcing depth, and per-candidate *relevance* and *evidence-support* rates for precision, each with a 500-resample bootstrap 95% confidence interval. This separation is deliberate: as §VII shows, a system can lead on depth and trail on precision, and that decomposition makes the benchmark diagnostic.

VI. Experimental Setup

Research questions. We ask whether requirements-driven sourcing reaches qualified people that keyword and embedding baselines miss, and whether grounded *satisfaction*, not retrieval, is the binding open challenge. We refine this into six questions (RQs):

- **RQ1** (*coverage at scale*) Over all 691 requirements, can systems surface candidates, and how do retrieval strategies compare on breadth and yield?
- **RQ2** (*robustness*) Is that breadth uniform across difficulty, task-type, and language, or concentrated on easy cases?
- **RQ3** (*complementarity*) Does requirements-driven retrieval surface people the rest of the field *misses*?
- **RQ4** (*grounded satisfaction*) Is grounded *satisfaction*, not retrieval, the binding open challenge, and does the benchmark isolate and reliably validate its measurement?
- **RQ5** (*efficiency*) Is the requirements-driven design cost-effective in retrieval latency and judging cost?
- **RQ6** (*elicitation*) Does the bounded clarification interview recover latent requirement constraints that a single-shot reading misses?

Benchmark. From 1,500 generated requirements, CADA admits 1,007 as well formed, classifying 691 as *open* and 316 as *closed* (§V-A). The 316 closed requirements are used *only* to validate the open/closed boundary and are *not* run

against any system under test: all empirical results use the 691 open requirements, spanning ten difficulty levels, sixteen sourcing task-types (e.g., career-trajectory, cross-verification), and seven languages. For graded scoring, we build evidence-backed acceptance oracles for 203 unique open requirements (mean pool 44.6 people, range 6–67, mean reliability 0.60). We evaluate two axes: *breadth* over **open691**, all 691 requirements (RQ1–RQ3), and *grounded quality*, where the CGPS judge grades every candidate from all 21 systems against the oracle and we report recall of relevant real people with bootstrap confidence intervals (RQ4).

Systems compared. On open691, we compare **21 configurations** in four families: (i) *product* pipelines, including ⌑ DINQ TalentScout and five deployed engines (Exa-search, two code-search variants, and two commercial people-search APIs); (ii) *autonomous-web* LLMs that self-orchestrate browsing (GPT-, Gemini-, DeepSeek-, Grok-class); (iii) *Exa-augmented* LLMs given a neural-web tool (adding GLM- and MiniMax-class); and (iv) *bare* LLMs with no retrieval. A source ablation runs ⌑ DINQ TalentScout's retrieval backbone in three modes: *internal* (a proprietary people index), *web* (neural search), and *mix*, over 100 requirements.

Scope and honesty. The benchmark gives each requirement *directly*, exercising the agent's *retrieval, verification, and delivery* stages but *not* its interactive elicitation (§IV-C), an architectural contribution for which we make no empirical claim (a controlled user study is the foremost future work, §IX). We report breadth *and* grounded-quality metrics for all 21 configurations, so sourcing depth and per-candidate verification are credited separately (§VII-D). **Metrics:** *coverage* (fraction of requirements with $\geq$ 1 candidate) and *yield* (mean candidates/requirement) measure breadth; *complementarity* is normalized-name overlap against pooled baselines. For grounded quality, *recall* is the share of the union pool of relevant real people a system surfaces, *reachable yield* counts relevant candidates with a fetchable URL, and per-candidate *relevance* and *evidence-support* rates measure precision, all computed by the CGPS judge with bootstrap 95% confidence intervals (§V-D).

VII. Results

A. RQ1: Coverage and Yield at Scale

Sourcing first requires returning someone. Across *all* 691 *open requirements* and 21 configurations, ⌑ DINQ TalentScout is the only system that pairs complete coverage with deep yield: it covers *every* requirement (100%) and returns the most candidates (49.9/req.) by unioning a proprietary people index with neural web search. Coverage alone is nearly saturated, with one autonomous GPT-class LLM also reaching 100%, at *one-third* the yield (19.8), but below the top, retrieval is the bottleneck: Exa-augmented LLMs cover 96–99% yet return only 4–12 candidates, and bare/Grok-class models miss up to 45% of requirements.

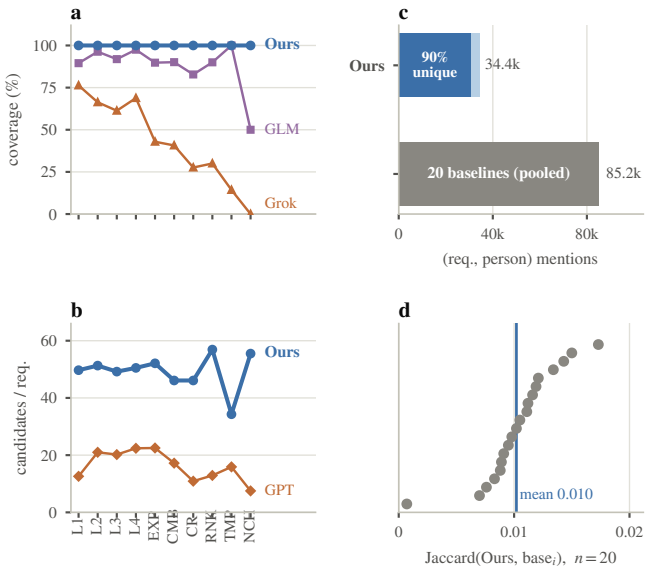

Fig. 3. **Breadth (RQ2) and complementarity (RQ3) over all** 691 **requirements.** *(a,b)* across the ten difficulty levels, ⌑ DINQ TalentScout holds 100% coverage and a flat ∼50 candidates per requirement, while a bare LLM sags and a Grok-class web agent collapses on the hard levels. *(c)* of the 34.4k (requirement, person) mentions ⌑ DINQ TalentScout returns, 90.0% (30,963) appear in *none* of 20 pooled baselines, whose union is 2.5× larger. *(d)* mean Jaccard between ⌑ DINQ TalentScout and each baseline is ≈0.01, near-orthogonal.

B. RQ2: Robust Across Difficulty, Type, and Language

Top-line coverage can mask success only on easy cases. On the full 691 set (Fig. 3a,b), ⌑ DINQ TalentScout is *invariant*: 100% coverage and ∼ 46–57 candidates across all ten difficulty levels, all sixteen sourcing task-types, and all seven languages. Hard levels (cross-verify, combination, ranking) and non-English requirements impose no loss. Baselines *degrade structurally*: the Grok-class model collapses on cross-verify (28%), temporal (14%), niche (0%), and Chinese (38%), while bare LLMs sag on career-trajectory and stability requirements (20–50%). The gap is largest on niche-stability and temporal task-types: every baseline either collapses or thins to a handful of candidates, while ⌑ DINQ TalentScout still provides 100% coverage at 34–56 candidates. The workflow template and full-coverage guard (§IV) make the agent exhaust the requirement rather than stop at easy hits. Thus, near-100% headline breadth does not predict long-tail breadth; only the engineered agent remains usable.

> ► **Finding 2.** ⌑ DINQ TalentScout's breadth is *invariant*: 100% coverage and ∼50 candidates across all ten difficulty levels, sixteen task-types, and seven languages, while baselines degrade structurally on hard (cross-verify, temporal, niche) and non-English requirements; robust breadth is a property of the engineered agent, not of scale alone.

C. RQ3: ⌑ DINQ TalentScout Surfaces People the Whole Field Misses

Yield is valuable only when it adds people the field does not already find. We pooled candidate names from 20 baselines and measured overlap with ⌑ DINQ TalentScout's output (Fig. 3c;

normalized-string matching measures coverage/overlap, *not* correctness). Of the 34,406 (requirement, person) mentions ⌑ DINQ TalentScout returns over 691 requirements, 90.0% (30,963) **are surfaced by none of the** 20 **baselines combined**, although their union (85,174) is 2.5× larger than ⌑ DINQ TalentScout's pool. Mean pairwise Jaccard is 0.010, *near-orthogonal* to the field, and a strict, name-order/diacritic-robust re-match changes the unique fraction only to 89.7%. Even pooling the field (6.24× the best single system) misses what ⌑ DINQ TalentScout adds. This says *who is found*, not who is correct (scoring follows in §VII-D); exact-string matching also makes uniqueness an *upper bound* when surface forms differ (e.g., Han characters vs. pinyin).

Why the orthogonality? ⌑ DINQ TalentScout's *hybrid* evidence base changes the candidate universe. By category and evidence source, its *mix* retrieval beats a web-only (Exa) backbone in every category (academic, company, GitHub, and Hugging Face), by +5.2 to +11.3 unique people per requirement. The sources are also *disjoint*: web-only baselines cite LinkedIn for essentially 100% of their evidence, whereas ⌑ DINQ TalentScout grounds many candidates in Scholar, GitHub, and Hugging Face profiles beyond a LinkedIn-only crawl. Different sources produce different people, yielding the near-orthogonality of Fig. 3d, the central breadth result.

► **Finding 3.** ⌑ DINQ TalentScout is *near-orthogonal* to the entire field: 90% of the 34k (requirement, person) mentions it returns appear in *none* of 20 pooled baselines (mean Jaccard 0.010), and pooling all 20 (6.2× the best single system) still misses what ⌑ DINQ TalentScout adds: requirements-driven retrieval contributes a distinct, complementary slice of the people-universe.

D. RQ4: ⌑ DINQ TalentScout Recalls the Most Relevant Real People

⌑ DINQ TalentScout is the strongest sourcing engine on TalentTrace: it recalls the largest pool of relevant real people, with a statistically decisive margin. We grade every candidate from all 21 configurations with the CGPS judge over the 691 requirements; a candidate is a relevant real person iff the judge does not mark it off-topic. This uses name, organization, and title, so concise candidates without thick evidence are not penalized.

Naive single-candidate metrics mislead (verbosity bias, depth penalties, unstable hallucination calls, circular gold pools), so we measure recall of relevant real people over a method-neutral union pool, with reachable yield and a bootstrap 95% confidence interval per system.

⌑ DINQ TalentScout recalls 0.241 of the union pool of 93,444 relevant real people (Fig. 4), 1.9× the next system, an autonomous GPT-class web agent at 0.126. A 500-resample bootstrap gives ⌑ DINQ TalentScout a 95% CI of $[0.230, 0.251]$, disjoint from the second system's $[0.120, 0.133]$ and from all 20 baselines' intervals, so the lead is not sampling noise. ⌑ DINQ TalentScout also surfaces the most reachable relevant candidates (14,267) and the most relevant people per requirement (32.8), 1.9× the next. It ranks first on every composite quality score, with $F_1 = 0.365$, and also leads on F_2 and F_3, which weight recall more heavily. ⌑ DINQ TalentScout is first on *every* sourcing-depth board (Table II), winning the most relevant people head-to-head on 68% of requirements versus 14% for the next; its per-requirement depth stays essentially flat from the simple to the constraint-heavy strata (Fig. 4d), a lead that does not erode with difficulty.

TABLE II
⌑ DINQ TalentScout (**Ours**) on the 21-system CGPS sourcing-depth leaderboards. ⌑ DINQ TalentScout ranks first on *every* board, most by ∼2×; ⋆ denotes a bootstrap 95% CI disjoint from all 20 baselines.

Leaderboard	**Ours**	Next	Rank
Sourcing depth: Ours is #1			
Recall of relevant real people	**0.241**	0.126	1/21⋆
Relevant people / requirement	**32.8**	17.1	1/21
Reachable relevant (count)	**14,267**	11,795	1/21
Per-query win rate	**68%**	14%	1/21
F_1, F_2, F_3 composite	**#1**	—	1/21
Simple / heavy-constraint strata	**#1**	—	1/21

⌑ DINQ TalentScout best serves deep sourcing by producing the widest real, reachable candidate pool, while precision-ranking LLMs serve as complementary verifiers that return few but accurate candidates, a natural division of labor.

► **Finding 4.** ⌑ DINQ TalentScout significantly recalls the largest pool of relevant real people on TalentTrace, making it the best *sourcing* engine, complementary to precision-focused verifiers.

Human evaluation. To complement the automated metrics, 30 professional headhunters compared the slates ⌑ DINQ TalentScout and several systems under test returned for real TalentTrace requirements and picked the best by their *overall professional judgment* of quality (a holistic preference, no fixed rubric). ⌑ DINQ TalentScout ranked first for 25 of the 30, ahead of GPT-5.5 and GLM-5.2: the practitioners who would use it prefer the deeper slate. GLM-5.2 is a revealing exception: third with the headhunters yet low automatically (0.062–0.073 recall vs. ⌑ DINQ TalentScout's 0.241), because its small, *focused* set is penalized by the recall-oriented benchmark but reads well to raters judging the shortlist they *see*; on the practitioner-facing measure, ⌑ DINQ TalentScout wins outright.

E. RQ5: Efficiency and Cost

Requirements-driven sourcing is cheap when computation is spent only on hard decisions. The hybrid backbone is the fastest *grounded* option (0.92 s to first result, 38% faster than web-only) because it runs both sources in parallel and short-circuits. The commit–defer cascade skips the expensive arbitrator on 95.3–97.5% of requirement-kind decisions; a `no-early-exit` control gives identical verdicts at ∼20× the cost. Oracle construction is a one-time $347.77 over 203 requirements ($1.71 each).

Ablations. **Mix retrieval** dominates a 100-requirement source ablation: web-only loses 7.6 people/requirement and the internal index's fact-checkable evidence, and internal-only drops to 56% coverage. The **two-stage commit**, **kickoff recipe**, and **full-coverage guard** are mechanistically motivated; isolating each is future work.

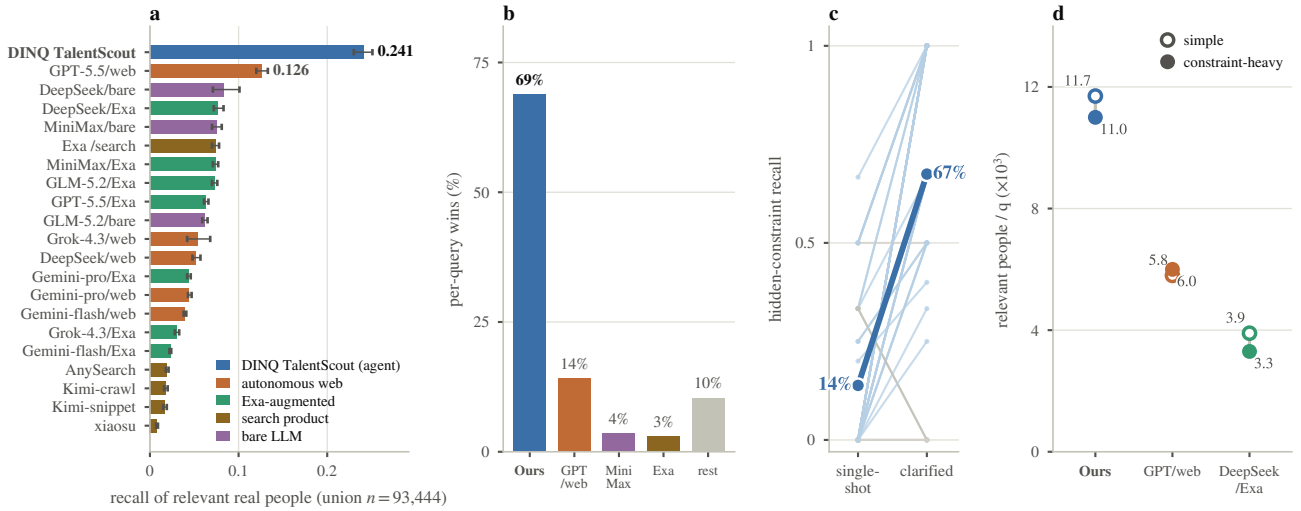

Fig. 4. **Grounded quality (RQ4) and requirement elicitation (RQ6).** *(a)* relevant-real-person recall over the method-neutral union pool, all 21 systems, with bootstrap 95% CIs: ⌑ DINQ TalentScout recalls 0.241 (1.9× the next), with an interval disjoint from every one of the 20 baselines. *(b)* head-to-head, ⌑ DINQ TalentScout returns the most relevant people on 68% of the 691 requirements, versus 14% for the next. *(c)* the elicitation ablation (RQ6): each thin line is one of 40 requirements (light blue improves, grey ties or regresses); the bold line is the mean recall of *hidden* latent constraints, 13.8% → 67.5% under the bounded clarification interview (4.9×; +0.54, 95% CI [+0.41, +0.65]; 33/40 improve). *(d)* the depth lead is robust to difficulty: across the *simple* and *constraint-heavy* strata, ⌑ DINQ TalentScout's relevant people per query stays essentially flat and remains ∼2× the next system on both (RQ2).

F. RQ6: Does Clarification Recover Latent Requirements?

TalentTrace issues each requirement single-shot, so we test ⌑ DINQ TalentScout's clarification contract (§IV-C) separately, in a *controlled elicitation ablation* (Fig. 4c) with a simulated oracle-user (the standard protocol in elicitation research [18]) on $N{=}40$ requirements. For each, we decompose the requirement into atomic *gold* constraints (mean 6.2), hide a random half to form an under-specified first request, and measure how many *hidden* (latent) constraints each condition recovers. NO-ELICIT reads the request once; ELICIT runs the bounded interview: a small budget of targeted questions (the contract of §IV-C) a truthful oracle-user answers from the full requirement. Clarifier/extractor, oracle-user, and constraint-judge are *three distinct models*, ruling out same-model artifacts.

The bounded interview recovers 4.9× more hidden latent constraints than single-shot reading (Fig. 4c; 13.8% → 67.5%, 33 of 40 requirements improve), and overall constraint capture rises from 0.29 to 0.41 (+0.12, CI [+0.03, +0.20]). The gain holds across *every* constraint dimension (Fig. 5), and is largest exactly where single-shot reading fails most: geography (0% → 75%), organization type (4% → 48%), and recency (9% → 65%): the implicit "where / what kind / when" constraints a vague request omits. That the gains concentrate on precisely the dimensions a recruiter leaves unstated, rather than on arbitrary ones, is evidence of *interview competence* [18]: the bounded clarifier spends its question budget on the most under-determined axes, the behaviour a well-formed elicitation contract should induce. This isolates and validates the elicitation step that the single-shot benchmark, by construction, cannot.

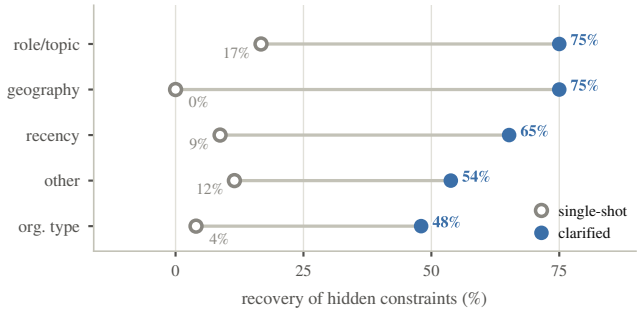

Fig. 5. **RQ6: clarification recovers latent constraints on every dimension.** Per-dimension recovery of the *hidden* constraints, single-shot reading vs. the bounded interview ($N{=}40$). The interview lifts recovery on every dimension, most steeply on geography (0% → 75%) and organization type (4% → 48%), the implicit "where/what-kind" constraints a vague request almost never states.

► **Finding 6.** A bounded clarification interview recovers 67.5% of the latent requirement constraints that single-shot reading misses (13.8%), a 4.9× gain (95% CI [+0.41, +0.65], 33/40 improve), and the gain holds on *every* constraint dimension (largest on geography, 0% → 75%), empirically validating ⌑ DINQ TalentScout's elicitation contract.

VIII. DISCUSSION AND IMPLICATIONS FOR RE

Acceptance as an RE diagnostic. Decomposing a deliverable into "found the right people" (recall) versus "grounded the evidence" (precision and evidence-support), and then into relevance and verification stages (§V-D), makes acceptance a requirements-traceability instrument: it localizes *where* a requirement fails: retrieval, when the right people were never surfaced, or verification, when surfaced candidates cannot be

justified. But the acceptance criteria are requirements too, and an unexamined assumption about admissible evidence can silently change a verdict; TalentTrace therefore makes criteria validation first-class, with auditable rubrics (§V-B) and deterministic scoring. More broadly, LLM-as-judge evaluation of SE artifacts should validate the evaluator, not merely apply it.

Recall is the right sourcing objective. In requirements-engineering terms, an open sourcing request denotes a predicate over the world, and the deliverable is the *extension* of that predicate, not a short list of individually safe items. For a headhunter, the bottleneck is surfacing the deepest set of true, reachable candidates: precision is a cheap downstream human filter, while a missed candidate is unrecoverable. This makes recall–precision a division of labor. ⌑ DINQ TalentScout is the *sourcing* engine, leading recall by $1.9\times$; precision-ranking LLMs are *verifiers* that prune candidates after the sourcing obligation is met. On that obligation, ⌑ DINQ TalentScout's lead is decisive: head-to-head it returns the most relevant people on 68% of all 691 requirements (Fig. 4b), versus 14% for the next system, and holds a $\sim 2\times$ margin on both the simple and the constraint-heavy halves (RQ2).

Is the lead just a bigger index? A natural worry is that ⌑ DINQ TalentScout's recall reflects a proprietary index, not its requirements-driven design. We separate the two: the recall margin *is* sourcing depth, and we claim only that a deep sourcing engine beats query-rankers on coverage of the predicate's extension. The *requirements* contributions are index-independent: the clarification contract recovers $4.9\times$ more latent constraints using *no* index (RQ6), TalentTrace scores any system under one methodology, and the sourcing-vs-verifier reframing is conceptual, so ⌑ DINQ TalentScout proves a requirements-driven agent can be the deepest sourcing engine, not that the index is the contribution.

One principle, four constraints. The design is deliberately *symmetric*: a tool-using agent diverges along four axes (under-determined input, exploratory process, fast-but-incomplete output, and a possibly non-terminating loop), and ⌑ DINQ TalentScout applies one requirements constraint to each (clarification contract, workflow template, two-stage commit, bidirectional termination guards). The contribution is the symmetry itself: requirements discipline turns an open-ended agent into a bounded, verifiable, reproducible one.

The judge cannot manufacture the lead. An LLM judge could inflate an agent's apparent quality only if the protocol rewarded that agent's style. TalentTrace avoids this circularity: relevance is decided from structured fields such as name, organization, and title, not evidence thickness; recall is computed over a *method-neutral* union pool rather than an LLM-built gold set; and the same judge scores all 21 systems with the same rubric. A 500-resample bootstrap places ⌑ DINQ TalentScout's interval disjoint from every baseline. Remaining judge noise is real, especially hallucination calls on concise candidates, but it bears on precision estimates; it does not explain ⌑ DINQ TalentScout's recall lead.

Generality and transfer. Neither the four-constraint principle nor the generate→validate→oracle→verify harness is specific to people: both apply wherever an agent must turn an under-determined request into a bounded, verifiable result (literature and dataset discovery, vendor selection, compliance evidence). Candidate sourcing is the *demanding* case, with open-set oracles and adversarial evidence (profiles, paywalls, stale pages), so a method that closes the loop here should transfer to easier ones.

IX. Threats to Validity

Construct validity. Requirement satisfaction is multi-faceted, so we report complementary metrics (recall and reachable yield for depth, per-candidate relevance and evidence-support for precision) with bootstrap 95% CIs rather than one score. The judge (one LLM, applied identically to all 21 systems) decides *relevance* from structured fields and is reliable, but its *truthfulness* judgments are noisy for concise profile candidates; we therefore treat precision/evidence results as approximate and the recall lead, which does not depend on them, as robust. Scoring is *deterministic* (the LLM emits only labels) over a method-neutral pool.

Internal validity. First, our evaluation oracles are model-constructed: cross-family, evidence-grounded, and left `null` when unsupported (mean reliability 0.60), but not exhaustively human-verified at scale; individual pool members may be stale or missing. Second, all evaluation uses the *open* requirements: the 316 closed validate only the open/closed boundary, are never run against systems under test, and support *no* closed-requirement claims (the CGPS closed-set F1 path is a scorer capability we do not exercise here). Third, absolute scores depend on scorer configuration (an early version structurally penalized profile-style evidence, which we corrected), so the robust finding is the *relative* ordering, not the absolute level; a stricter fact-crawling scorer exists in our code but was not run. Finally, the still-settling values are coverage for no-retrieval *bare* configurations, which were converging upward at submission and should be read as lower bounds.

Scope of the elicitation evidence. The main benchmark issues each requirement single-shot; we therefore evaluate the clarification contract separately, in a controlled ablation with a *simulated* oracle-user (RQ6). This isolates whether the interview recovers latent constraints, but does not measure live human-interview UX or user effort; a human-subject study remains future work.

External validity. Our population is web-accessible people; though the benchmark spans seven languages, evidence skews toward English and profile sites, so conclusions may not transfer to closed enterprise corpora or under-documented populations. Absolute numbers depend on evolving models and a fixed baseline set, so longitudinal use would require periodic re-grounding of the baseline pool.

X. Related Work

Requirement-driven agents and recruiting. Closest to ours is work on *requirement-driven* LLM agents for software engineering: rewriting issue descriptions into structured repair specifications [19] or interleaving tool use to fix code [20],

[21]. These studies show that RE principles sharpen agents, but for *code*, not people. In people search, expertise retrieval, reviewer matching [1], [2], [22], and LLM recruiting tools [3] treat the request as a *finished* query and optimize relevance to it. Concurrent people-search benchmarks keep this stance: [23] grades platforms on 119 *fixed* queries by criteria-grounded nDCG and [24] ranks tools by human-preference Elo, both scoring *retrieval against a given query*, with no elicitation, no well-formedness check, and no recall of real people against an oracle. ⌑ DINQ TalentScout fills the empty cell in an *authors-the-requirement* vs. *query-given* × *interactive* vs. *one-shot* space: to our knowledge, it is the first system to *author and refine a people-requirement interactively* from a vague request, then verify the delivered people against it.

Elicitation and LLMs for RE. Interviews (progressive, question-driven clarification of implicit needs) are foundational to elicitation [5], [25]; their core challenge is ambiguity and tacit knowledge [6], [14], [26]. Recent work automates the interviewer with LLMs [15], [27]–[31] and, concurrently with us, benchmarks interview *competence* [18], but stops at *producing* a requirement. We close the loop: ⌑ DINQ TalentScout elicits and *validates* a sourcing requirement, then *verifies* delivered people against an evidence oracle encoding IEEE 830 / ISO·IEC·IEEE 29148 quality attributes [10], [17] as operational criteria (Table I): sourcing as RE for retrieval-grounded, set-valued deliverables [32].

Acceptance criteria and the test-oracle problem. Verification imports the test-oracle problem [8] into requirement satisfaction: open requirements are *non-testable* [9], so we use a *derived, partial, probabilistic* oracle and treat acceptance criteria as the sourcing analogue of a user story's [33], [34]. To our knowledge, this is the first connection between the oracle problem and people-finding, and the first to make *validating the evaluator* (Boehm's V&V [13] applied to the oracle itself via the adequacy obligation [4], [12]) a benchmark obligation.

Agentic search and clarification. Browsing and deep-research benchmarks [35], [36] use *fully specified* queries with one verifiable answer, sidestepping the ambiguity we invert: an open people-requirement has many valid answers and *no* gold set. Clarifying questions for under-specified needs (the anomalous state of knowledge [7]) are studied in IR [37], [38] but not tied to the requirements lifecycle. Like recent SE work [39], we adapt heterogeneous multi-agent debate [40] into a deterministic, early-exiting validator that mitigates LLM-judge bias [41], over a guarded reason–act loop [42], [43].

XI. Conclusion

Open-ended candidate sourcing is fundamentally a requirements-engineering problem, not a retrieval one: the binding difficulty is eliciting, validating, and verifying an under-determined people-requirement, not ranking against a finished query. We made this case with ⌑ DINQ TalentScout (an interactive agent that converges an LLM's divergence on four axes through one requirements-discipline principle) and TalentTrace, a benchmark that runs the requirements lifecycle. Across 21 systems and all 691 requirements, ⌑ DINQ TalentScout *recalls the most relevant real people of any system* (1.9× the next, bootstrap-significant), positioning it as a deep *sourcing* engine and precision-ranking LLMs as complementary *verifiers*. The four-constraint design and its generate–validate–oracle–verify harness transfer to any under-determined agentic task.

Data availability. The system studied in this paper is proprietary; due to confidentiality restrictions, its source code and data cannot be publicly released. All algorithms and settings needed for re-implementation are specified in the paper (§IV–§V and Alg. 1–2).

References

[1] K. Balog, Y. Fang, M. De Rijke, P. Serdyukov, L. Si *et al.*, "Expertise retrieval," *Foundations and Trends® in Information Retrieval*, vol. 6, no. 2–3, pp. 127–256, 2012.

[2] K. Balog, L. Azzopardi, and M. De Rijke, "Formal models for expert finding in enterprise corpora," in *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*, 2006, pp. 43–50.

[3] C. Gan, Q. Zhang, and T. Mori, "Application of llm agents in recruitment: A novel framework for resume screening. arxiv," *arXiv preprint arXiv:2401.08315*, 2024.

[4] P. Zave and M. Jackson, "Four dark corners of requirements engineering," *ACM transactions on Software Engineering and Methodology (TOSEM)*, vol. 6, no. 1, pp. 1–30, 1997.

[5] D. Zowghi and C. Coulin, "Requirements elicitation: A survey of techniques, approaches, and tools," in *Engineering and managing software requirements*. Springer, 2005, pp. 19–46.

[6] A. Ferrari, P. Spoletini, and S. Gnesi, "Ambiguity and tacit knowledge in requirements elicitation interviews," *Requirements Engineering*, vol. 21, no. 3, pp. 333–355, 2016.

[7] N. J. Belkin, R. N. Oddy, and H. M. Brooks, "Ask for information retrieval: Part i. background and theory," *Journal of documentation*, vol. 38, no. 2, pp. 61–71, 1982.

[8] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[9] E. J. Weyuker, "On testing non-testable programs," *The Computer Journal*, vol. 25, no. 4, pp. 465–470, 1982.

[10] "Iso/iec/ieee international standard - systems and software engineering – life cycle processes – requirements engineering," *ISO/IEC/IEEE 29148:2018(E)*, pp. 1–104, 2018.

[11] M. Glinz, "On non-functional requirements," in *15th IEEE international requirements engineering conference (RE 2007)*. IEEE, 2007, pp. 21–26.

[12] C. A. Gunter, E. L. Gunter, M. Jackson, and P. Zave, "A reference model for requirements and specifications," *IEEE Software*, vol. 17, no. 3, pp. 37–43, 2000.

[13] B. W. Boehm, "Verifying and validating software requirements and design specifications," *IEEE software*, vol. 1, no. 1, p. 75, 1984.

[14] O. Zaremba and S. Liaskos, "Towards a typology of questions for requirements elicitation interviews," in *2021 IEEE 29th International Requirements Engineering Conference (RE)*. IEEE, 2021, pp. 384–389.

[15] Y. Shen, A. Singhal, and T. Breaux, "Requirements elicitation follow-up question generation," in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*. IEEE, 2025, pp. 117–129.

[16] Anthropic, "Claude Agent SDK for Python," https://github.com/anthropics/claude-agent-sdk-python, 2025.

[17] "Ieee recommended practice for software requirements specifications," *IEEE Std 830-1998*, pp. 1–40, 1998.

[18] D. Jin, Z. Jin, Z. Fang, L. Li, X. Yang, Y. He, and X. Chen, "Reqelicitgym: An evaluation environment for interview competence in conversational requirements elicitation," *arXiv preprint arXiv:2602.18306*, 2026.

[19] S. Kuang, Z. Tian, K. Lin, C. Tao, S. Wang, H. Bai, L. Shang, and J. Chen, "Reagent: Requirement-driven llm agents for software issue resolution," *arXiv preprint arXiv:2604.06861*, 2026.

[20] I. Bouzenia, P. Devanbu, and M. Pradel, "Repairagent: An autonomous, llm-based agent for program repair," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 2188–2200.

[21] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1592–1604.

[22] D. Mimno and A. McCallum, "Expertise modeling for matching papers with reviewers," in *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2007, pp. 500–509.

[23] W. Wang, T. Shi, S. Zhang, B. Xia, Z. Xie, C. Zeng, Q. Zhang, L. Ai, Y. Yu, K. Zhang *et al.*, "Peoplesearchbench: A multi-dimensional benchmark for evaluating ai-powered people search platforms," *arXiv preprint arXiv:2603.27476*, 2026.

[24] V. Slaykovskiy, M. Zvegintsev, Y. Sakhonchyk, and H. Ajamian, "Evaluating ai recruitment sourcing tools by human preference," *arXiv preprint arXiv:2504.02463*, 2025.

[25] C. Palomares, X. Franch, C. Quer, P. Chatzipetrou, L. López, and T. Gorschek, "The state-of-practice in requirements elicitation: an extended interview study at 12 companies," *Requirements engineering*, vol. 26, no. 2, pp. 273–299, 2021.

[26] I. Hadar, P. Soffer, and K. Kenzi, "The role of domain knowledge in requirements elicitation via interviews: an exploratory study," *Requirements Engineering*, vol. 19, no. 2, pp. 143–159, 2014.

[27] A. Korn, S. Gorsch, and A. Vogelsang, "Llmrei: Automating requirements elicitation interviews with llms," in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*. IEEE, 2025, pp. 19–30.

[28] Y. Shen and T. Breaux, "Stakeholder preference extraction from scenarios," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 69–84, 2024.

[29] D. Jin, Z. Jin, X. Chen, and C. Wang, "Mare: Multi-agents collaboration framework for requirements engineering," *arXiv preprint arXiv:2405.03256*, 2024.

[30] S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. El Mansi, S. P. Erdeniz, and V.-M. Le, "Leveraging llms for the quality assurance of software requirements," in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 2024, pp. 389–397.

[31] C. Arora, J. Grundy, and M. Abdelrazek, "Advancing requirements engineering through generative ai: Assessing the role of llms," in *Generative AI for Effective Software Development*. Springer, 2024, pp. 129–148.

[32] A. Vogelsang and M. Borg, "Requirements engineering for machine learning: Perspectives from data scientists," in *2019 IEEE 27th international requirements engineering conference workshops (REW)*. IEEE, 2019, pp. 245–251.

[33] M. Cohn, *User stories applied: For agile software development*. Addison-Wesley Professional, 2004.

[34] B. Haugset and G. K. Hanssen, "Automated acceptance testing: A literature review and an industrial case study," in *Agile 2008 Conference*. IEEE, 2008, pp. 27–38.

[35] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese, "Browsecomp: A simple yet challenging benchmark for browsing agents," *arXiv preprint arXiv:2504.12516*, 2025.

[36] Y. Huang, L. F. Ribeiro, M. Hardalov, B. Dhingra, M. Dreyer, and V. Saligrama, "Deepfact: Co-evolving benchmarks and agents for deep research factuality," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 34 356–34 386.

[37] M. Aliannejadi, H. Zamani, F. Crestani, and W. B. Croft, "Asking clarifying questions in open-domain information-seeking conversations," in *Proceedings of the 42nd international acm sigir conference on research and development in information retrieval*, 2019, pp. 475–484.

[38] S. Rao and H. Daumé III, "Learning to ask good questions: Ranking clarification questions using neural expected value of perfect information," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018, pp. 2737–2746.

[39] X. Du, G. Zheng, K. Wang, Y. Zou, Y. Wang, W. Deng, J. Feng, M. Liu, B. Chen, X. Peng *et al.*, "Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag," *ACM Transactions on Software Engineering and Methodology*, 2024.

[40] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 11 733–11 763.

[41] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in neural information processing systems*, vol. 36, pp. 46 595–46 623, 2023.

[42] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[43] Anthropic, "Building effective agents," https://www.anthropic.com/engineering/building-effective-agents, 2025.